

Design Patterns

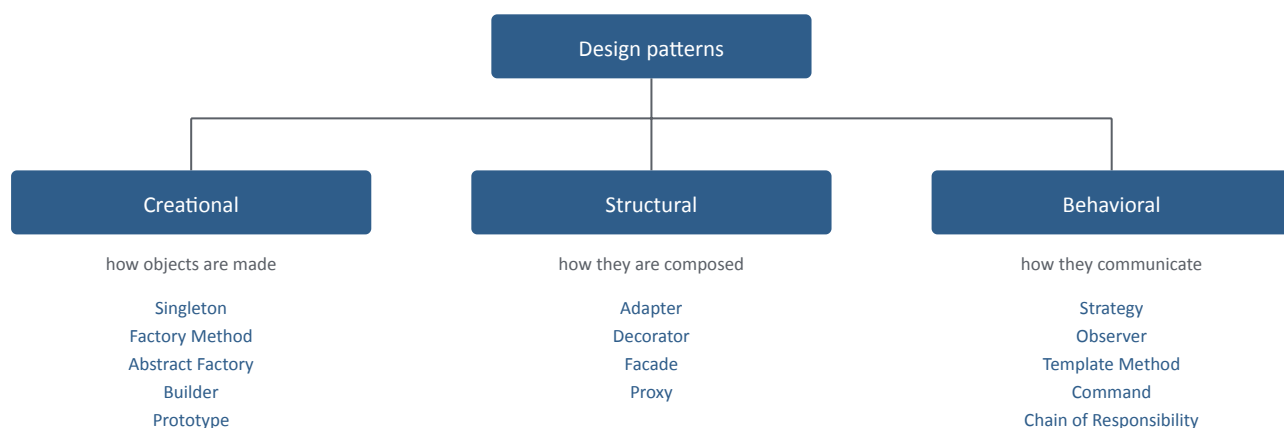
A design pattern is a named solution to a problem that appears again and again. Half the value is the name: "use a builder here" is a complete instruction between two engineers.

The Gang of Four book cataloged twenty-three of them in 1994, in three groups.

A reading order:

The three groups → Singleton → Factory Method → Abstract Factory
→ Builder → Prototype → Structural → Behavioral → Where they already are

1. The three groups



Creational patterns answer "how is this built", structural "how do these fit together", behavioral "who talks to whom".

A pattern is a description, not a library. Recognizing one already present in a framework is worth more than adding one, because most of the patterns a working developer meets were written by somebody else.

The failure to avoid is reaching for a pattern because it is a pattern. An interface with one implementation, a factory that returns one type and a strategy that never varies are all the shape of a pattern with none of the benefit.

2. Singleton

One instance for the whole application, and a global way to reach it.

The simple version is created when the class loads:

```
public final class AppConfig {

    private static final AppConfig INSTANCE = new AppConfig();

    private AppConfig() { }

    public static AppConfig getInstance() {
        return INSTANCE;
    }
}
```

That is thread safe, because the class loader guarantees the field is assigned once before any thread can see it. It is also created whether or not anything uses it.

Creating it on first use needs care:

```
public final class AppConfig {

    private static volatile AppConfig instance;

    private AppConfig() { }

    public static AppConfig getInstance() {
        if (instance == null) { // no lock in the common case
            synchronized (AppConfig.class) { // check again, holding the lock
                if (instance == null) {
                    instance = new AppConfig();
                }
            }
        }
        return instance;
    }
}
```

volatile is not optional there. Without it a second thread can see the reference assigned before the constructor has finished, and use an object whose fields are still empty. That is double-checked locking, and the missing **volatile** is the classic bug in it.

The enum version has neither problem:

```
public enum AppConfig {
    INSTANCE;

    private String theme = "dark";

    public String getTheme() { return theme; }
}
```

Thread safe, correct across serialization, and reflection cannot create a second one.

Singleton is the most criticized pattern in the book. It is global state under another name: it hides a dependency from the constructor, and tests share whatever it holds. A Spring `@Service` gives the same single instance while still arriving through the constructor, which is why application code rarely needs this pattern directly.

3. Factory Method

Move the choice of which class to create out of the caller.

```
public abstract class NotificationService {

    protected abstract Notification create();    // the factory method

    public void notify(String message) {
        create().send(message);
    }
}

public class EmailNotificationService extends NotificationService {
    @Override
    protected Notification create() {
        return new EmailNotification();
    }
}
```

The version used far more often is a **static factory method**, which is not the Gang of Four pattern but carries the same name in conversation:

```
public static Money rupees(long amount) {
    return new Money(amount, Currency.getInstance("INR"));
}
```

It beats a constructor in three ways. It has a name, so two factories with the same parameter types can coexist. It can return a cached instance rather than a new one. And it can return a subtype, so the caller depends on the interface.

`Integer.valueOf`, `List.of` and `Optional.of` are all this.

4. Abstract Factory

A factory that creates a family of related objects, so the family stays consistent.

```

public interface UIFactory {
    Button    createButton();
    Checkbox createCheckbox();
}

public class DarkUIFactory implements UIFactory {
    public Button    createButton()    { return new DarkButton(); }
    public Checkbox createCheckbox() { return new DarkCheckbox(); }
}

```

The caller holds one `UIFactory` and cannot produce a dark button beside a light checkbox. That guarantee is what the pattern is for.

Factory Method → one product, chosen by a subclass
 Abstract Factory → a family of products that must match

5. Builder

Build an object step by step, when a constructor would need too many parameters.

```

public final class Employee {

    private final String name;
    private final String email;
    private final BigDecimal salary;
    private final LocalDate hiredOn;

    private Employee(Builder builder) {
        this.name    = builder.name;
        this.email    = builder.email;
        this.salary   = builder.salary;
        this.hiredOn  = builder.hiredOn;
    }

    public static Builder builder() {
        return new Builder();
    }
}

```

The builder itself carries the defaults and the validation:

```

public static final class Builder {

    private String name;
    private String email;
    private BigDecimal salary = BigDecimal.ZERO;
    private LocalDate hiredOn = LocalDate.now();

    public Builder name(String name)    { this.name = name; return this; }
    public Builder email(String email)  { this.email = email; return this; }
    public Builder salary(BigDecimal s) { this.salary = s; return this; }

    public Employee build() {
        if (name == null || email == null) {
            throw new IllegalStateException("name and email are required");
        }
        return new Employee(this);
    }
}

```

```

Employee employee = Employee.builder()
    .name("Ravi")
    .email("ravi@example.com")
    .salary(new BigDecimal("75000"))
    .build();

```

What it replaces is the telescoping constructor:

```

new Employee("Ravi", "ravi@example.com", salary, hiredOn, null, null, true, false);

```

Nobody can read that call, and swapping two arguments of the same type compiles.

Validation belongs in `build()`, so an invalid `Employee` never exists at all. Checking after construction leaves a window where it does.

Since Java 16 a record covers the simple cases. A builder earns its place when there are many optional fields, or when the object must be immutable and validated.

6. Prototype

Create a new object by copying an existing one, when building from nothing is expensive.

```
public final class ReportTemplate {

    private final String title;
    private final List<Section> sections;

    public ReportTemplate(ReportTemplate other) {    // a copy constructor
        this.title = other.title;
        this.sections = new ArrayList<>(other.sections);
    }
}
```

Cloneable is the version to avoid. It is an interface with no methods, `Object.clone` is `protected`, and the default it provides is a shallow copy that shares every referenced object with the original. A copy constructor or a static `copyOf` does the same job in plain code.

Copy	What happens to a referenced list
Shallow	Both objects share the same list. A change shows in both
Deep	The copy gets its own list, and its own elements

The example above is between the two: a new list holding the same `Section` objects. That is correct when `Section` is immutable and wrong when it is not, which is the decision the pattern actually asks for.

Use it when construction is expensive and many near-identical objects are needed: a parsed template, a configured client, a prepared query.

7. Structural patterns

These four are about fitting objects together.

Adapter makes one interface look like another, so two things that were not designed to meet can.

```
public class LegacyPaymentAdapter implements PaymentClient {

    private final LegacyGateway legacy;

    @Override
    public PaymentResult charge(Money amount) {
        int paise = amount.toPaise();    // the old API's units
        return toResult(legacy.doPayment(paise));
    }
}
```

Decorator adds behavior by wrapping an object, without touching its class and without subclassing. The JDK shows it in one line:

```
Reader reader = new BufferedReader(
    new InputStreamReader(
        new FileInputStream(file)));
```

Each wrapper adds one thing and passes the rest through. A subclass for every combination would be unmanageable; wrappers compose.

Facade puts one simple entry point over a complicated subsystem.

```
public class CheckoutFacade {

    public void placeOrder(Order order) {
        inventory.reserve(order);
        payment.charge(order);
        shipping.schedule(order);
        notifications.confirm(order);
    }
}
```

The caller learns one method instead of four services.

Proxy stands in for another object and controls access to it. Spring's `@Transactional` works this way, and so does a lazily loaded JPA association: the field holds a proxy, and touching it triggers the query.

Pattern	It solves
Adapter	Two interfaces that do not match
Decorator	Adding behavior without subclassing
Facade	A subsystem that is hard to call
Proxy	Controlling access to an object

Adapter and Decorator both wrap. An adapter changes the interface; a decorator keeps it and adds to the behavior.

8. Behavioral patterns

These are about how objects communicate.

Strategy swaps an algorithm at run time.

```
public interface ShippingCost {
    BigDecimal calculate(Order order);
}

ShippingCost free      = order -> BigDecimal.ZERO;
ShippingCost byWeight = order -> order.weight().multiply(RATE);
```

Since Java 8 a strategy is usually a lambda. `Comparator` is the one every developer already uses.

Observer tells many objects when one thing changes.

```
@Component
public class OrderPlacedListener {

    @EventListener
    public void on(OrderPlaced event) {
        inventory.reserve(event.orderId());
    }
}
```

Spring's event publisher is this pattern. The publisher does not know who is listening, which is what lets a listener be added without touching it.

Template Method fixes a sequence and lets subclasses fill in the steps.

```
public abstract class ImportJob {

    public final void run() {           // final: the order is not negotiable
        Data data = read();
        validate(data);
        write(data);
    }

    protected abstract Data read();
    protected abstract void write(Data data);

    protected void validate(Data data) { } // optional hook
}
```

Marking `run` final is the pattern. Without it a subclass can replace the sequence, which is the one thing the base class was protecting.

Command turns a request into an object, so it can be queued, logged or undone.

```
public interface Command {
    void execute();
}

public record CancelOrder(Long orderId, OrderService service) implements Command {
    public void execute() { service.cancel(orderId); }
}
```

Anything that puts work on a queue is doing this, including `Runnable` itself.

Chain of Responsibility passes a request along a line of handlers until one deals with it. A servlet filter chain is the everyday example: each filter either handles the request or passes it on.

9. Where they already are

Most patterns a working developer meets are in the libraries rather than in the application.

Pattern	Already in
Singleton	Spring beans, <code>Runtime.getRuntime()</code>
Factory Method	<code>Integer.valueOf</code> , <code>List.of</code> , <code>Optional.of</code>
Builder	<code>StringBuilder</code> , <code>HttpRequest.newBuilder</code> , <code>Stream.builder</code>
Prototype	Copy constructors, <code>List.copyOf</code>
Adapter	<code>Arrays.asList</code> , <code>InputStreamReader</code>
Decorator	<code>BufferedReader</code> , <code>Collections.unmodifiableList</code>
Facade	<code>JdbcTemplate</code> , <code>RestClient</code>
Proxy	<code>@Transactional</code> , JPA lazy loading, dynamic proxies
Strategy	<code>Comparator</code> , any functional interface
Observer	<code>ApplicationEventPublisher</code> , <code>PropertyChangeListener</code>
Template Method	<code>AbstractList</code> , <code>JdbcTemplate</code>
Chain of Responsibility	Servlet filters, the Spring Security filter chain

Naming the pattern in code we did not write is the useful skill. It explains why `BufferedReader` takes a `Reader`, why `@Transactional` stops working on a call from inside the same class, and why a `Comparator` is passed rather than subclassed.

The pieces together

Pattern	In one line
Singleton	One instance, reachable globally
Factory Method	A subclass decides which class to create
Abstract Factory	A family of products that must match
Builder	Step by step construction, validated at the end
Prototype	A new object copied from an existing one
Adapter	Makes one interface look like another
Decorator	Adds behavior by wrapping
Facade	One entry point over a subsystem
Proxy	Controls access to an object
Strategy	Swaps an algorithm at run time
Observer	Tells many objects when one changes
Template Method	A fixed sequence with steps left open
Command	A request as an object
Chain of Responsibility	Handlers in a line, until one acts

Consider an order service. `Order` is built with a builder, because it has four optional fields and must be valid once created. `ShippingCost` is a strategy, supplied as a lambda. Placing an order publishes an event, and three listeners react, which is observer. The payment provider is reached through an adapter, so replacing it changes one class. None of that was chosen because it is a pattern; each is the plain answer to a problem, and the name came afterwards.

Open a class from a framework we already use and name the pattern it is. `Comparator`, `BufferedReader` and `@Transactional` are three good places to start.